but it is doing so in part by using its own prediction about future returns (since the training data will include its own prediction).

Listing 5.9. *N*-step training with CartPole

```
def run_episode(worker_env, worker_model, N_steps=10):
    raw_state = np.array(worker_env.env.state)
    state = torch.from_numpy(raw_state).float()
    values, logprobs, rewards = [],[],[]
    done = False
    j=0
    G=torch.Tensor([0])                          1
    while (j < N_steps and done == False):       2
        j+=1
        policy, value = worker_model(state)
        values.append(value)
        logits = policy.view(-1)
        action_dist =
torch.distributions.Categorical(logits=logits)
        action = action_dist.sample()
        logprob_ = policy.view(-1)[action]
        logprobs.append(logprob_)
        state_, _, done, info =
worker_env.step(action.detach().numpy())
        state = torch.from_numpy(state_).float()
        if done:
            reward = -10
            worker_env.reset()
        else:                                    3
            reward = 1.0
            G = value.detach()
        rewards.append(reward)
    return values, logprobs, rewards, G
```

- ***1*** The variable G refers to the return. We initialize to 0.
- ***2*** Plays game until N steps or when episode is over
- ***3*** If episode is not done, sets return to the last state value

The only things we've changed are the conditions for the `while` loop (exit by *N* steps), and we've set the return to be the state value of the last step if the episode is not over, thereby enabling bootstrapping. This new `run_episode` function explicitly returns `G`, the return, so to get this to work we need to make a couple minor updates to the `update_params` function and the `worker` function.